開発仕様書 / 支払い実行サービス（Payout Execution Service）

作成日: 2026-03-18 | 最終更新: 2026-03-18 | 作成者: Claude (Hori Shinnosuke 依頼)
開発対象: 後工程 / 支払い実行サービス（Payout Execution Service）
初回ユースケース: Twomi クリエイター報酬支払い（Payoneer経由）

【用語定義】

前工程 / 後工程　　: 処理の段階・フローを指す呼称
収益管理システム　　: 前工程のシステムを指す呼称（収益計算・残高管理・UI等）
支払い実行サービス　: 後工程のシステムを指す呼称（本書の開発対象）

日本語版

---

0. 前提情報

プロジェクト名: 支払い実行サービス（Payout Execution Service）
オーナー（Biz）: Hori Shinnosuke
リリース対象: MVP
関係者: PM(JP) / Eng(JP+VN) / Finance
外部サービス: Payoneer Mass Payout API

本書の読み方:
本書は支払い実行サービスの開発仕様書です。
支払い実行サービスとは、収益管理システムから支払いリストを受け取り、Payoneerを通じて実際の送金を実行し、結果を収益管理システムに返すサービスです。
収益管理システムの仕様は「インターフェース定義（入力仕様）」として参照情報として記載します。

将来の拡張方針（考慮事項）:
支払い実行サービスは将来的に複数のシステムから横断利用することを想定して設計します。
ただしMVPでは「Twomi クリエイター報酬支払い」の1ユースケースに集中して実装します。
横断利用を見越し、呼び出し元のシステム識別（source_service）と、プロバイダー部分の差し替えが可能な内部構造（Adapterパターン）を最初から組み込んでおきます。

1. システム全体の構造と支払い実行サービスの位置づけ

前工程 / 収益管理システム（Revenue Management System） ※開発対象外

役割:
・購入検知・収益計算・残高管理・出金申請受付・支払いリスト生成
・全ビジネスデータの所有者（支払い実行結果も含む）
・支払い実行サービスからの結果を受け取りDBに記録
・クリエイターへの通知 / UI（ダッシュボード・出金申請・履歴）

↓ ① 支払いリストを渡す

後工程 / 支払い実行サービス（Payout Execution Service） ← 本書の開発対象

役割:
・収益管理システムから支払いリストを受け取る
・Payoneer Mass Payout APIを呼び出して送金実行
・Payoneerからの実行結果（成功/失敗）を受信
・② 実行結果を収益管理システムに返す
制約:
・ビジネスデータを自身では保持しない
・送金ロジック以外の判断をしない

↓ 送金リクエスト / 結果受信

Payoneer（外部送金サービス）

---

2. 支払い実行サービスの責務（開発スコープ）

支払い実行サービスが「やること」:
・収益管理システムから受け取った支払いリストをPayoneer Mass Payout APIに送信する
・Payoneerからのレスポンスを受信する
・PayoneerからのWebhookを受信・検証する
・各支払いの実行結果（成功/失敗/エラー詳細）を収益管理システムに返す
・冪等性の保証（同一request_idの二重送信防止）
・Payoneer APIの認証・トークン管理
・リトライ処理（タイムアウト・一時エラー時）
・Webhookが届かない場合のポーリングによる結果補完

支払い実行サービスが「やらないこと」:
・収益計算（誰にいくら払うかの判断）→ 収益管理システムの責務

・バリデーション（残高・KYC・最低出金額チェック）→ 収益管理システムの責務
・クリエイターへの通知 → 収益管理システムの責務
・支払いデータの永続管理 → 収益管理システムの責務
・ビジネスルールに基づく判断 → 収益管理システムの責務

3. 収益管理システム→支払い実行サービス インターフェース定義（入力仕様）

支払い実行サービスが収益管理システムから受け取るデータ。
支払いリスト（1件あたり）:
  request_id　string 必須　収益管理システムの出金申請ID（idempotency keyとして使用）
  source_service　string 必須　呼び出し元システム識別子（例: "twomi_creator_payout"）
  payoneer_email　string 必須　支払い先のPayoneerアカウントメールアドレス
  amount　decimal 必須　支払い金額
  currency　string 必須　通貨コード（例: JPY / USD）
  description　string 任意　説明文（例: "Twomi creator payout - {request_id}"）

複数件をまとめてバッチで受け取ることも可。

呼び出し方式（要確認・設計）:
  方式A. 同期REST API: 収益管理システムが支払い実行サービスのエンドポイントにPOSTする
  方式B. 非同期キュー: 収益管理システムがキュー（SQS等）に積み、支払い実行サービスがポーリング
  ※どちらの方式を採用するかは収益管理システムチームとのインターフェース合意が必要

4. 支払い実行サービス→収益管理システム インターフェース定義（出力仕様）

支払い実行サービスが収益管理システムに返す実行結果（1件ごと）:
  request_id　string　収益管理システムの出金申請ID（識別に使用）
  status　string　"completed" / "failed"
  payoneer_payout_id　string　Payoneer側の支払いID（参照用。収益管理システムが保存）
  processed_at　datetime　Payoneerが処理した日時
  error_code　string　失敗時のみ。エラーコード
  error_message　string　失敗時のみ。エラー詳細

返却タイミング:
  ① Payoneer APIレスポンスで即時返却できる場合 → その場で収益管理システムに返す
  ② Payoneer処理が非同期の場合（Webhook経由）
    → PayoneerからWebhookを受信次第、収益管理システムに転送

→ Webhookが一定時間届かない場合 → ポーリングで結果を取得し収益管理システムに通知

5. Payoneer API仕様

認証: OAuth 2.0 Client Credentials
ベースURL: https://api.payoneer.com（本番） / サンドボックスURL（開発・テスト用）

主要エンドポイント:
  [1] 認証トークン取得
  POST /v2/oauth2/token
  パラメータ: client_id, client_secret, grant_type=client_credentials

  [2] 支払い実行
  POST /v4/programs/{program_id}/payouts
  リクエストボディ:
    client_reference_id: "{request_id}" ← idempotency key
    payee.id: "{payoneer_email}"
    amount.value: 900.00
    amount.currency: "JPY"
    description: "Twomi creator payout - {request_id}"

  [3] 支払い結果照会（ポーリング用）
  GET /v4/programs/{program_id}/payouts/{payout_id}

Webhook受信:
  受信するイベント:
    PAYMENT_COMPLETED → 送金成功
    PAYMENT_FAILED → 送金失敗
    PAYEE_NOT_FOUND → 支払い先が見つからない
  Webhookの検証:
  Payoneerから送られるHMACシグネチャを検証し、正規リクエストであることを必ず確認する。

program_id について（要確認）:
  既存のアバタープロバイダーへの支払い用プログラムIDと統合するか、新規プログラムを作成するか要確認。

6. 支払い実行サービスの処理フロー詳細

[収益管理システムから支払いリストを受け取る]
  ↓
[request_idの重複チェック（処理中の二重リクエスト防止）]
  ├── 処理中の重複 → 409 Conflict を返して終了

```
  └── 問題なし ↓
[PayoneerのOAuth2.0トークンを取得（キャッシュ活用）]
 ↓
[Payoneer Mass Payout APIに送金リクエストを送信]
  ├── タイムアウト / 一時エラー → 指数バックオフで最大3回リトライ
  │     3回失敗 → 収益管理システムにエラー通知（status: failed, error_code:
PAYONEER_TIMEOUT）
  └── レスポンス受信
      ├── 即時完了 → 結果を収益管理システムに返す
      └── 非同期処理中（pending）→ Webhook受信待ち
          ↓
        [Webhookを受信・シグネチャ検証]
         ├── PAYMENT_COMPLETED      → 収益管理システムに success を通知
         └── PAYMENT_FAILED
           PAYEE_NOT_FOUND       → 収益管理システムに failure + error_code を通知

[Webhookが一定時間届かない場合のフォールバック]
  → ポーリング: Payoneer GET APIで定期的に結果を照会し、確定次第収益管理システムに通
知
```

7. 冪等性の保証

目的: 同じ出金申請が誤って二重送信された場合でも二重決済が発生しないようにする。
実装方法:
・request_idをPayoneer APIのclient_reference_id（idempotency key）として使用
・Payoneer側: 同一client_reference_idのリクエストは1回のみ処理（Payoneerの仕様として保証）
・支払い実行サービス側: 処理中のrequest_idを一時保持し、重複リクエストを即時拒否する（409 Conflict）
※TTL付き（例: 30分）で保持。完了・失敗後は自動削除。ビジネスデータの永続管理ではない。

8. エラーハンドリング仕様

Payoneer APIタイムアウト
→ 対応: 指数バックオフで最大3回リトライ
→ 収益管理システムへの通知: failed + PAYONEER_TIMEOUT

Payoneer認証エラー
→ 対応: トークン再取得して1回リトライ
→ 収益管理システムへの通知: failed + AUTH_ERROR（要管理者対応）

PAYEE_NOT_FOUND（宛先アカウントなし）

→ 対応: リトライなし
→ 収益管理システムへの通知: failed + PAYEE_NOT_FOUND

INSUFFICIENT_FUNDS（Payoneerプール残高不足）
→ 対応: リトライなし
→ 収益管理システムへの通知: failed + INSUFFICIENT_FUNDS（要管理者対応）

Webhookシグネチャ不正
→ 対応: リクエストを破棄・ログ記録
→ 収益管理システムへの通知: なし（セキュリティログのみ）

Webhookタイムアウト
→ 対応: ポーリングにフォールバック
→ 収益管理システムへの通知: ポーリング結果で通知

request_id重複（処理中）
→ 対応: 即時拒否
→ 収益管理システムへの通知: 409 Conflict

9. セキュリティ要件

・PayoneerのAPIキー/シークレットは環境変数（シークレットマネージャ）で管理。コード埋め込み禁止
・WebhookエンドポイントはHMACシグネチャ検証を必須実装（IPホワイトリストは任意の追加対策）
・全Payoneer API通信のアクセスログ・エラーログを記録
・TLS 1.2以上での通信

10. 非機能要件

可用性: 99.9%以上
受付応答タイム（収益管理システムへの受付応答）: 5秒以内
送金完了（Payoneer処理含む）: 通常5営業日以内（Payoneer SLA準拠）
ログ保持期間: 最低1年
Webhookエンドポイントタイムアウト: 30秒

11. MVP範囲確定

MVPで「絶対にやる」:
・Payoneer OAuth2.0認証・トークン管理
・支払いリスト受信 → Payoneer Mass Payout API送信

・APIレスポンス受信・パース
・Webhook受信エンドポイント実装 + HMACシグネチャ検証
・Webhookタイムアウト時のポーリングフォールバック
・実行結果を収益管理システムに返却
・冪等性保証（request_idによる重複処理防止）
・リトライ処理（指数バックオフ、最大3回）
・source_serviceによる呼び出し元識別・ログ記録
・エラーハンドリング（各エラー種別ごとの収益管理システムへの通知）

将来の拡張（MVP対象外）:
・複数支払いプロバイダー対応（銀行振込・Stripe等の追加）
・呼び出し元システムごとのレートリミット
・Admin UI

内部設計の方針:
将来のプロバイダー追加・横断利用を見越し、Adapterパターンでプロバイダー部分を分離して実装する。
ただしMVPではPayoneerのみ実装。

12. 受け入れ条件（Acceptance Criteria）

【正常系: 支払い成功フロー】
Given: 収益管理システムから有効な支払いリスト（request_id, payoneer_email, amount, currency）が届く
When: 支払い実行サービスがPayoneer Mass Payout APIに送信する
Then: Payoneerが送金を実行し、支払い実行サービスが成功結果を収益管理システムに返す
（status: completed, payoneer_payout_id付き）

【異常系: Payoneer APIタイムアウト】
Given: Payoneer APIが応答しない
When: 送金リクエストを送信
Then: 指数バックオフで最大3回リトライ後、収益管理システムにエラー通知
（status: failed, error_code: PAYONEER_TIMEOUT）

【異常系: PAYEE_NOT_FOUND】
Given: 支払い先のPayoneerアカウントが存在しない
When: 送金リクエストを送信
Then: リトライなし。即座に収益管理システムにエラー通知
（status: failed, error_code: PAYEE_NOT_FOUND）

【異常系: Webhookが届かない】
Given: Payoneerが送金を実行したがWebhookが一定時間届かない
When: ポーリングタイマーが起動する
Then: Payoneer GET APIでポーリング実行。結果確定後に収益管理システムに通知

【冪等性】
Given: 同一request_idのリクエストが2回届く
When: 2回目のリクエストを支払い実行サービスが受信
Then: 2回目はブロック（409 Conflict）。Payoneerへの送信は1回のみ。

完了基準:
・Payoneer Sandbox環境でのE2Eテスト全ケース通過
・本番Payoneer APIとの疎通確認

13. 変更ルール定義

変更承認者: Hori Shinnosuke
変更時に必ず確認すること:
・収益管理システム→支払い実行サービスのインターフェース（入力仕様）への影響
・支払い実行サービス→収益管理システムのインターフェース（出力仕様）への影響
・冪等性・エラーハンドリングへの影響
・Payoneer API仕様との整合性

レビュー方法: PR + Confluence仕様書更新（VNチームはdev_customでBen経由共有）

14. 未定義事項リスト（開発着手前に確認が必要）

以下は本仕様書の策定時点で未確定の事項です。
「推奨」は参考案であり、最終決定は関係者での合意が必要です。

A. 通信・インターフェース

A-1. 収益管理システム→支払い実行サービスの呼び出し方式
状態: 未定（方式A: 同期REST API / 方式B: 非同期キュー）
影響: エントリーポイント・エラー応答・デプロイ構成が変わる
推奨: 方式A（同期REST API）をMVPでは採用。
将来の大量バッチ処理が必要になった段階でキュー方式へ移行。

A-2. 支払い実行サービス→収益管理システムへの結果返却方式
状態: 未定（同期返却 / Webhookコールバック / キュー書き込み）
影響: 出力インターフェースの実装全体
推奨: A-1で方式Aを採用した場合は同期返却。
Payoneer非同期時は収益管理システムのコールバックエンドポイントを呼び出す方式を推奨。
収益管理システム側にコールバック受信エンドポイントの実装が必要。

A-3. 収益管理システム→支払い実行サービスへのインバウンド認証
状態: 未定義（現仕様書に記載なし）
影響: 不正な第三者からの呼び出しを防ぐセキュリティ
推奨: APIキー（Bearer Token）をMVPでは採用。
シークレットマネージャーで管理し、Authorization: Bearer {api_key} で検証。

A-4. WebhookエンドポイントのURL・Payoneerへの登録手順
状態: 未定義（エンドポイントURLが存在しない）
影響: Payoneer非同期フローのテスト・本番稼働が不可能
推奨: PES自身がエンドポイントをホスト。
デプロイ後にPayoneer管理画面でURLを登録。開発着手時にサンドボックスでの登録手順も確認すること。

B. Payoneer / 外部サービス

B-1. Payoneer program_id
状態: 未確定（既存アバタープロバイダー用と統合 / 新規作成）
影響: 全Payoneer API呼び出しのURLパラメータ。未確定ではサンドボックスも使えない
推奨: 新規プログラムを作成。
クリエイター向けとプロバイダー向けを分離することで管理・レポーティング・トラブル対応が明確になる。

B-2. Payoneer OAuthトークンのリフレッシュ戦略
状態: トークン取得フローは定義済みだが、期限切れ・再取得ロジックが未定義
影響: 本番稼働中にトークン期限切れで送金が失敗するリスク
推奨: 有効期限の5分前に自動再取得（クロックベース）を採用。
401レスポンス検知時も即時再取得→1回リトライ（AUTH_ERRORハンドリングと統合）。

B-3. payoneer_emailによるPayee識別の前提確認
状態: payee.idにメールアドレスを渡す前提だが、Payoneer API実挙動未確認
影響: コア支払いロジックの正確性
推奨: サンドボックスで事前検証すること。payee.typeフィールドの追加要否もあわせて確認。

B-4. Webhookポーリングの間隔・上限回数
状態: 未定
影響: ポーリングフォールバックのAcceptance Criteriaを満たせない
推奨: 間隔5分・最大12回（合計1時間）を提案。
上限到達後は収益管理システムにPOLLING_TIMEOUTとして通知し人的確認フローへ。

C. インフラ・実装方式

C-1. 冪等性ストレージの実装方式
状態: 「処理中のrequest_idを一時保持」と定義済みだが実装方式が未定
影響: クラッシュ時のリカバリ方針・インフラ依存関係
推奨: Redis（TTL付き）を採用。
インメモリはクラッシュ時にロストするリスクがある。Redisは既存インフラ流用も可能。

D. 運用・監視

D-1. INSUFFICIENT_FUNDSエラー時のアラート通知
状態: 「要管理者対応」と記載されているが、通知方法が未定義
影響: 本番環境でエラーが積み上がっても誰にも通知されない
推奨: Slackの管理者チャンネルへの即時通知を採用。
監視ツール（Datadog / CloudWatch等）のアラート連携も検討。

D-2. 出金処理の頻度（バッチサイズ・レートリミット設計）
状態: 未定（即時申請制 / 週次バッチ / 月次バッチ）
影響: バッチサイズ・Payoneer APIレートリミット対応の設計
推奨: MVPでは即時申請制（1件ずつ処理）とし、バッチ処理は将来対応。
Payoneer APIのレートリミット仕様は事前確認が必要。

D-3. 収益計算のbase amount定義（App Store差引前/後）
状態: 未定（収益管理システム側の決定事項）
影響: 支払い実行サービスはバリデーションを行わないため直接影響は軽微。
収益管理システム側での確定後、入力仕様のamountフィールドの定義を明確化すること。
推奨: 支払い実行サービスはamountをそのまま送金する設計を維持。
金額の正確性は収益管理システムの責務とする。

【参考】収益管理システムの仕様概要（入力データの背景理解用・開発対象外）

収益分配モデル（アバター購入例: ¥3,000）:
・App Store / Google Play手数料 (30%): ¥900
・Twomi運営手数料 (40%): ¥1,200
・クリエイター収益 (30%): ¥900
※ base amountの定義（App Store差引前/後）は別途確定

出金ルール:
・最低出金額: ¥5,000
・支払いサイクル: 月末締め・翌月末払い
・出金方式: クリエイターがダッシュボードから申請（即時申請制、MVPではバッチ処理なし）

収益管理システムが管理するデータ（支払い実行サービスは保持しない）:
・購入データ、収益レコード、出金申請レコード、クリエイターPayoneer口座情報
・支払い実行サービスから受け取った実行結果（収益管理システムのDBに記録）

収益管理システムの収益ステータス（参考）:
  Pending（収益発生）
    ↓ 返金期間経過後
  Available（出金可能）
    ↓ 出金申請受付後
  Locked（処理中）
    ↓ 支払い実行サービスが成功結果を返す　↓ 支払い実行サービスが失敗結果を返す
  Paid（完了）　　Available（残高を戻す）

English Version

Development Spec / Payout Execution Service
Created: 2026-03-18 | Author: Claude (Requested by Hori Shinnosuke)
Development Target: Post-Process / Payout Execution Service
Initial use case: Twomi Creator Payout (via Payoneer)

Terminology:
  Pre-Process / Post-Process　　: Labels for processing stages/flow
  Revenue Management System (RMS)　: The pre-process system (revenue calc, balance mgmt, UI)
  Payout Execution Service (PES)　: The post-process system (this document's development target)

0. About This Document

This document is the development spec for the Payout Execution Service (PES).
PES receives a Payout List from the Revenue Management System (RMS), executes actual transfers via Payoneer, and returns execution results to RMS.
The RMS spec is included only as interface reference.

Future consideration:
PES is designed to be reusable across multiple internal systems.

For MVP, focus is on the Twomi Creator Payout use case only.
To support future cross-system usage, PES includes caller identification (source_service) and an internal Adapter pattern to allow provider replacement later.

1. System Architecture

Pre-Process / Revenue Management System (RMS) — NOT in scope

Responsibilities:
· Purchase detection, revenue calculation, balance management
· Withdrawal request intake, Payout List generation
· Owns ALL business data (including payout results)
· Creator notifications / UI (dashboard, withdrawal, history)

↓ ① Pass Payout List

Post-Process / Payout Execution Service (PES) ← THIS DOC'S TARGET

Responsibilities:
· Receive Payout List from RMS
· Call Payoneer Mass Payout API to execute transfers
· Receive execution results (success/failure) from Payoneer
· ② Return results to RMS
Constraints:
· Does NOT own business data
· Makes no business decisions beyond transfer logic

↓ Transfer request / Receive result

Payoneer (External transfer service)

---

2. PES Responsibilities

What PES DOES:
· Receive Payout List from RMS
· Call Payoneer Mass Payout API to execute transfers
· Receive API responses and Webhooks from Payoneer
· Return per-payment results (success/failure/error) to RMS
· Guarantee idempotency (prevent double-processing of same request_id)

· Manage Payoneer API auth and token lifecycle
· Retry logic (on timeouts / transient errors)
· Polling fallback when Webhooks are delayed

What PES does NOT do:
· Revenue calculation → RMS responsibility
· Validation (balance, KYC, minimum amount) → RMS responsibility
· Creator notifications → RMS responsibility
· Persistent data management → RMS responsibility

3. RMS → PES Interface (Input)

Per payment record:

request_id	string	Required	RMS withdrawal ID (used as idempotency key)
source_service	string	Required	Caller system identifier (e.g., "twomi_creator_payout")
payoneer_email	string	Required	Payoneer account email of recipient creator
amount	decimal	Required	Payment amount
currency	string	Required	Currency code (e.g., JPY / USD)
description	string	Optional	e.g., "Twomi creator payout - {request_id}"

Invocation method (to be agreed with RMS team):
Option A: Synchronous REST API — RMS POSTs to PES endpoint
Option B: Async queue (SQS etc.) — RMS enqueues, PES consumes

4. PES → RMS Interface (Output)

Per payment result:

request_id	string	RMS withdrawal ID (for RMS lookup)
status	string	"completed" or "failed"
payoneer_payout_id	string	Payoneer's payout ID (reference only; RMS stores)
processed_at	datetime	When Payoneer processed the payment
error_code	string	(Failure only) Error code
error_message	string	(Failure only) Error details

Return timing:
① Synchronous Payoneer response → return to RMS immediately
② Async Payoneer processing
→ Forward to RMS when Webhook received
→ If Webhook delayed → poll Payoneer GET API and notify RMS on result

5. Payoneer API Specification

```
Auth: OAuth 2.0 Client Credentials
Key endpoints:
  [1] Token acquisition
  POST /v2/oauth2/token
  Params: client_id, client_secret, grant_type=client_credentials

  [2] Execute payment
  POST /v4/programs/{program_id}/payouts
  Body:
    client_reference_id: "{request_id}"   ← idempotency key
    payee.id: "{payoneer_email}"
    amount.value: 900.00
    amount.currency: "JPY"
    description: "Twomi creator payout - {request_id}"

  [3] Query result (polling)
  GET /v4/programs/{program_id}/payouts/{payout_id}

Webhooks:
  Events to handle:
    PAYMENT_COMPLETED   → transfer successful
    PAYMENT_FAILED      → transfer failed
    PAYEE_NOT_FOUND     → recipient not found
  Webhook verification: HMAC signature validation required on all incoming requests.

program_id: Confirm whether to share with existing avatar provider program or create new.
```

6. PES Processing Flow

```
[Receive Payout List from RMS]
  ↓
[Check request_id for duplicates]
  ├── In-flight duplicate → return 409 Conflict
  └── OK ↓
[Get Payoneer OAuth2.0 token (use cache)]
  ↓
[Send to Payoneer Mass Payout API]
  ├── Timeout / error → exponential backoff, max 3 retries
  │     Failed → notify RMS: failed + PAYONEER_TIMEOUT
  └── Response received
      ├── Sync result → return to RMS immediately
      └── Async (pending) → wait for Webhook
            ↓
```

```
[Receive + verify Webhook]
├── PAYMENT_COMPLETED     → notify RMS: success
└── PAYMENT_FAILED
    PAYEE_NOT_FOUND      → notify RMS: failure + error_code

[Webhook timeout fallback]
  → Poll Payoneer GET API → notify RMS when confirmed
```

7. Idempotency

· Use request_id as client_reference_id (Payoneer idempotency key)
· Payoneer: guarantees same client_reference_id is processed only once
· PES: temporarily stores in-flight request_ids (TTL ~30 min)
  to reject duplicates immediately (409 Conflict)
  Note: in-flight tracking only — not persistent business data storage

8. Error Handling

```
Payoneer API timeout
→ Action: Exponential backoff, max 3 retries
→ Notify RMS: failed + PAYONEER_TIMEOUT

Payoneer auth error
→ Action: Re-fetch token, retry once
→ Notify RMS: failed + AUTH_ERROR (needs admin attention)

PAYEE_NOT_FOUND
→ Action: No retry
→ Notify RMS: failed + PAYEE_NOT_FOUND

INSUFFICIENT_FUNDS
→ Action: No retry
→ Notify RMS: failed + INSUFFICIENT_FUNDS (needs admin attention)

Invalid Webhook signature
→ Action: Discard request, log security event
→ Notify RMS: none (security log only)

Webhook timeout
→ Action: Fall back to polling
→ Notify RMS: notify via polling result

Duplicate request_id (in-flight)
```

→ Action: Reject immediately
→ Notify RMS: 409 Conflict

9. Security

· API keys/secrets in environment variables / secret manager only. Never hardcode.
· Webhook endpoint: HMAC signature verification required (IP whitelist is optional additional layer)
· Log all Payoneer API communications
· TLS 1.2+

10. Non-Functional Requirements

Availability: 99.9%+
Response time (RMS acknowledgment): Within 5 seconds
Payment completion (including Payoneer processing): Within 5 business days (Payoneer SLA)
Log retention: Minimum 1 year
Webhook endpoint timeout: 30 seconds

11. MVP Scope

Must have:
· Payoneer OAuth2.0 auth & token management
· Payout List ingestion → Payoneer Mass Payout API call
· API response parsing
· Webhook endpoint + HMAC signature verification
· Polling fallback for delayed Webhooks
· Return results to RMS
· Idempotency guarantee (request_id duplicate prevention)
· Retry logic (exponential backoff, max 3)
· source_service identification and logging
· Per-error-type error handling

Future (not MVP):
· Multiple payment provider support
· Per-caller rate limiting
· Admin UI

Internal design note:
Implement Payoneer logic inside an Adapter class to allow future provider replacement

without changing the core service interface.

12. Acceptance Criteria

[Happy path]
  Given: Valid Payout List received from RMS
  When: PES sends to Payoneer
  Then: Payoneer executes transfer → PES returns success to RMS
        (status: completed + payoneer_payout_id)

[Payoneer API timeout]
  Given: Payoneer API unresponsive
  When: PES sends payment request
  Then: Max 3 retries with backoff → notify RMS: failed + PAYONEER_TIMEOUT

[PAYEE_NOT_FOUND]
  Given: Recipient Payoneer account doesn't exist
  When: PES sends payment request to Payoneer
  Then: No retry → immediately notify RMS: failed + PAYEE_NOT_FOUND

[Webhook not received]
  Given: Payoneer processed but Webhook not received within timeout
  When: Polling timer activates
  Then: Polling fallback → notify RMS when result confirmed

[Idempotency]
  Given: Same request_id received twice
  When: Second request received by PES
  Then: Second request blocked (409 Conflict). Payoneer called only once.

Completion criteria:
· Full E2E test suite passes in Payoneer Sandbox
· Production Payoneer API connectivity verified

13. Undefined Items (Must Confirm Before Development)

Items listed below are unresolved at the time of this spec.
"Recommendation" entries are proposals only; final decisions require stakeholder agreement.

A. Communication / Interface

A-1. RMS → PES invocation method
Status: Undefined (Option A: Synchronous REST API / Option B: Async queue)
Impact: Defines PES entry point, error response model, and deployment topology
Recommendation: Use Option A (Synchronous REST API) for MVP.
Migrate to queue-based if high-volume batch processing is needed later.

A-2. PES → RMS result delivery method
Status: Undefined (sync response / Webhook callback / queue write)
Impact: Determines the entire output interface implementation
Recommendation: If Option A chosen for A-1: return result synchronously in HTTP response.
For async Payoneer results: callback to an RMS webhook endpoint.
RMS will need to expose a callback endpoint.

A-3. Inbound authentication from RMS to PES
Status: Undefined (not specified in this document)
Impact: Security — prevents unauthorized callers from triggering payouts
Recommendation: API key (Bearer Token) for MVP.
Managed via secret manager, validated via Authorization: Bearer {api_key} header.

A-4. Webhook endpoint URL and Payoneer registration
Status: Undefined (no URL exists yet)
Impact: Async Payoneer flow cannot be tested or deployed without this
Recommendation: PES hosts the endpoint itself.
Register URL in Payoneer admin console after deployment.
Confirm sandbox registration procedure at project kickoff.

B. Payoneer / External Services

B-1. Payoneer program_id
Status: Unconfirmed (share with existing avatar provider program or create new)
Impact: Required in every Payoneer API URL — sandbox testing is blocked without it
Recommendation: Create a new program.
Separating creator and provider programs improves management, reporting, and incident response clarity.

B-2. Payoneer OAuth token refresh strategy
Status: Token acquisition defined; expiry and refresh logic are not specified
Impact: Risk of silent payment failures when token expires in production

Recommendation: Pre-expiry refresh (5 minutes before expiry, clock-based).
On 401 response: re-fetch token and retry once (integrate with AUTH_ERROR handler).

B-3. payoneer_email as Payee identifier — confirmation needed
Status: Assumed; actual Payoneer API behavior unverified
Impact: Correctness of core payment call
Recommendation: Verify in Payoneer sandbox before implementation.
Check whether payee.type field is also required.

B-4. Webhook polling interval and max retry count
Status: Undefined
Impact: Polling fallback acceptance criteria cannot be verified without this
Recommendation: Interval: 5 minutes, max: 12 attempts (1 hour total).
After max attempts: notify RMS as POLLING_TIMEOUT for manual follow-up.

C. Infrastructure / Implementation

C-1. Idempotency storage implementation
Status: "Temporarily store in-flight request_ids" defined; storage medium unspecified
Impact: Crash recovery behavior and infrastructure dependencies
Recommendation: Use Redis with TTL.
In-process memory is lost on crash; Redis provides durability
and is likely available in existing infrastructure.

D. Operations / Monitoring

D-1. INSUFFICIENT_FUNDS alert mechanism
Status: "Needs admin attention" noted; notification method undefined
Impact: Errors accumulate silently in production with no one alerted
Recommendation: Immediate Slack notification to admin channel.
Consider monitoring tool integration (Datadog / CloudWatch Alerts).

D-2. Withdrawal processing frequency
Status: Undefined (on-demand / weekly batch / monthly batch)
Impact: Batch size and Payoneer API rate limit design
Recommendation: On-demand (one at a time) for MVP; defer batch processing.
Verify Payoneer API rate limits before implementation.

D-3. Base amount definition (before/after App Store deduction)

Status: Undefined — RMS team decision
Impact: Minimal direct impact on PES (PES transfers whatever amount it receives).
Recommendation: PES maintains pass-through design — transfer the amount as received.
Amount accuracy is RMS's responsibility.
RMS must clarify the definition of the amount field after confirming base amount.

Reference: Revenue Management System Overview (context only — not in scope)

Revenue split model (example: ¥3,000 avatar purchase):
· App Store / Google Play fee (30%): ¥900
· Twomi platform fee (40%): ¥1,200
· Creator earnings (30%): ¥900
* Base amount definition (before/after App Store deduction) to be confirmed separately.

Withdrawal Rules:
· Minimum withdrawal amount: ¥5,000
· Payment cycle: Month-end close, payment by end of following month
· Withdrawal method: Creator requests via dashboard (on-demand, no batch processing in MVP)

RMS earnings status flow (reference):
Pending → Available → Locked
→ Paid (when PES returns success)
→ Available (when PES returns failure, balance restored)

Phiên bản tiếng Việt

Tài liệu phát triển / Dịch vụ thực hiện thanh toán (Payout Execution Service - PES)
Ngày tạo: 2026-03-18 | Tác giả: Claude (theo yêu cầu của Hori Shinnosuke)

Thuật ngữ:
Tiền xử lý / Hậu xử lý : Nhãn cho các giai đoạn xử lý
Hệ thống quản lý doanh thu (RMS) : Hệ thống tiền xử lý (tính doanh thu, quản lý số dư, UI)
Dịch vụ thực hiện thanh toán (PES) : Hệ thống hậu xử lý (mục tiêu của tài liệu này)

0. Giới thiệu

Tài liệu này là đặc tả phát triển cho PES (Dịch vụ thực hiện thanh toán).
PES nhận danh sách thanh toán từ RMS, thực hiện chuyển tiền qua Payoneer, và trả kết quả về cho RMS.

Lưu ý về mở rộng trong tương lai:
PES được thiết kế để có thể dùng chung cho nhiều hệ thống nội bộ trong tương lai.
Tuy nhiên với MVP, tập trung vào use case Twomi Creator Payout.
Tích hợp source_service để nhận dạng hệ thống gọi và dùng Adapter pattern để dễ thay thế nhà cung cấp sau này.

1. Kiến trúc hệ thống

Tiền xử lý / Hệ thống quản lý doanh thu (RMS) — Ngoài phạm vi

---

Trách nhiệm:
· Phát hiện giao dịch, tính doanh thu, quản lý số dư
· Tiếp nhận yêu cầu rút tiền, tạo Payout List
· Sở hữu TẤT CẢ dữ liệu nghiệp vụ
· Thông báo creator / UI

↓ ① Chuyển Payout List

Hậu xử lý / Dịch vụ thực hiện thanh toán (PES) ← MỤC TIÊU

Trách nhiệm:
· Nhận Payout List từ RMS
· Gọi Payoneer Mass Payout API thực hiện chuyển tiền
· Nhận kết quả thực hiện từ Payoneer
· ② Trả kết quả về RMS
Ràng buộc:
· Không lưu giữ dữ liệu nghiệp vụ
· Không đưa ra quyết định nghiệp vụ

↓ Yêu cầu chuyển tiền / Nhận kết quả

Payoneer (Dịch vụ chuyển tiền bên ngoài)

---

2. Đặc tả giao diện đầu vào (RMS → PES)

Mỗi bản ghi trong Payout List:

request_id	string	Bắt buộc	ID yêu cầu rút tiền của RMS (dùng làm idempotency key)
source_service	string	Bắt buộc	Tên hệ thống gọi (ví dụ: "twomi_creator_payout")
payoneer_email	string	Bắt buộc	Email tài khoản Payoneer của creator
amount	decimal	Bắt buộc	Số tiền
currency	string	Bắt buộc	Mã tiền tệ (JPY / USD...)
description	string	Tùy chọn	Mô tả giao dịch

3. Đặc tả giao diện đầu ra (PES → RMS)

Kết quả mỗi giao dịch:

request_id	string	ID yêu cầu rút tiền của RMS
status	string	"completed" hoặc "failed"
payoneer_payout_id	string	ID giao dịch Payoneer (chỉ tham chiếu)
processed_at	datetime	Thời điểm xử lý
error_code	string	Chỉ khi thất bại
error_message	string	Chỉ khi thất bại

4. Luồng xử lý

```
[Nhận Payout List từ RMS]
  ↓
[Kiểm tra trùng lặp request_id]
  ├── Đang xử lý trùng → trả về 409 Conflict
  └── OK ↓
[Lấy token Payoneer OAuth2.0 (dùng cache)]
  ↓
[Gửi đến Payoneer Mass Payout API]
  ├── Timeout / lỗi → backoff mũ, tối đa 3 lần thử lại
  │     Thất bại → thông báo RMS: failed + PAYONEER_TIMEOUT
  └── Nhận phản hồi
      ├── Kết quả đồng bộ → trả về ngay cho RMS
      └── Xử lý bất đồng bộ → chờ Webhook
            ↓
          [Nhận + xác thực Webhook]
          ├── PAYMENT_COMPLETED     → thông báo RMS: thành công
          └── PAYMENT_FAILED
              PAYEE_NOT_FOUND      → thông báo RMS: thất bại + error_code

[Dự phòng khi Webhook không đến]
  → Polling Payoneer GET API → thông báo RMS khi có kết quả
```

5. Mô hình phân chia doanh thu (tham khảo — RMS quản lý)

Ví dụ: mua avatar ¥3,000:
· Phí App Store / Google Play (30%): ¥900
· Phí nền tảng Twomi (40%): ¥1,200
· Thu nhập creator (30%): ¥900

6. Các vấn đề chưa được xác định (cần xác nhận trước khi phát triển)

Các mục dưới đây chưa được xác định tại thời điểm viết tài liệu này.
Mục "Đề xuất" chỉ là gợi ý; quyết định cuối cùng cần sự thống nhất của các bên liên quan.

A. Giao tiếp / Giao diện

A-1. Phương thức gọi RMS → PES: REST API đồng bộ hay hàng đợi bất đồng bộ
Đề xuất: Dùng REST API đồng bộ (Phương án A) cho MVP.

A-2. Phương thức trả kết quả PES → RMS: chưa xác định
Đề xuất: Trả đồng bộ nếu dùng Phương án A; callback Webhook nếu Payoneer xử lý bất đồng bộ.

A-3. Xác thực đầu vào từ RMS đến PES: chưa định nghĩa trong tài liệu hiện tại
Đề xuất: API key (Bearer Token) cho MVP.

A-4. URL Webhook endpoint và đăng ký với Payoneer: chưa xác định
Đề xuất: PES tự host endpoint; đăng ký URL sau khi triển khai trong Payoneer admin.

B. Payoneer / Dịch vụ bên ngoài

B-1. Program ID Payoneer: dùng chung hay tạo mới
Đề xuất: Tạo program mới (tách biệt creator và provider để dễ quản lý).

B-2. Chiến lược làm mới OAuth token: chưa định nghĩa
Đề xuất: Tự động lấy lại trước 5 phút khi hết hạn (clock-based).

B-3. Xác nhận payoneer_email là định danh payee: chưa kiểm tra thực tế với Payoneer API
Đề xuất: Xác minh trong môi trường sandbox trước khi triển khai.

B-4. Khoảng thời gian polling và số lần tối đa: chưa xác định
Đề xuất: 5 phút × 12 lần (tổng 1 giờ); sau đó thông báo POLLING_TIMEOUT cho RMS.

C. Hạ tầng

C-1. Phương thức lưu trữ idempotency: chưa xác định
Đề xuất: Redis với TTL (tránh mất dữ liệu khi dịch vụ khởi động lại).

D. Vận hành / Giám sát

D-1. Cơ chế cảnh báo khi INSUFFICIENT_FUNDS: chưa định nghĩa
Đề xuất: Thông báo ngay đến kênh Slack quản trị.

D-2. Tần suất xử lý rút tiền: chưa xác định
Đề xuất: Xử lý theo từng yêu cầu (on-demand) cho MVP.

D-3. Định nghĩa base amount (trước/sau phí App Store): thuộc trách nhiệm RMS
Đề xuất: PES chuyển đúng số tiền nhận được; xác nhận định nghĩa amount với RMS.

Quy tắc rút tiền:
· Số tiền rút tối thiểu: ¥5,000
· Chu kỳ thanh toán: Chốt cuối tháng, thanh toán cuối tháng tiếp theo
· Phương thức rút: Creator yêu cầu qua dashboard (xử lý theo từng yêu cầu, không xử lý hàng loạt trong MVP)